

■ Apache Spark

Top 50 Toughest Scenario-Based

Interview Questions

Asked at Netflix · Uber · LinkedIn · Databricks · Meta · Google · Amazon

50

Questions

7

Categories

FAANG+

Companies

Expert

Level

■ Table of Contents

Section 1	Performance Optimization & Tuning	Q1–Q8
Section 2	Memory Management & Spill	Q9–Q14
Section 3	Shuffles, Joins & Skew Handling	Q15–Q22
Section 4	Streaming & Fault Tolerance	Q23–Q30
Section 5	Delta Lake, Lakehouse & Storage	Q31–Q37
Section 6	Cluster Management & Scheduling	Q38–Q43
Section 7	Advanced APIs, SQL & Architecture	Q44–Q50

Section 1: Performance Optimization & Tuning

Q1

Your Spark job reads 10 TB of Parquet data daily but takes 4 hours. Profiling shows 80% of time is in a single wide transformation. How do you diagnose and fix this?

■ Asked at: Netflix · Databricks · Uber | ■ Difficulty: Hard

■ **Answer:**

Start by checking the Spark UI for the slow stage — look at shuffle read/write bytes, spill metrics, and task duration variance. 80% in one stage usually means data skew or an expensive shuffle. Steps: (1) Check if partition count is adequate — `spark.sql.shuffle.partitions` default 200 is too low for 10 TB; set to 2000-4000. (2) Use `.explain()` to verify the physical plan — look for BroadcastHashJoin vs SortMergeJoin. (3) Enable AQE (`spark.sql.adaptive.enabled=true`) to auto-coalesce small partitions and handle skew. (4) If Parquet columns are selectively read, verify predicate pushdown is happening. (5) Check for UDFs that block Catalyst optimization — replace with built-in functions.

■ **Key Code / Config:**

```
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.adaptive.skewJoin.enabled', 'true')
spark.conf.set('spark.sql.shuffle.partitions', '3000')
# Analyze plan
df.explain(mode='cost')
```

■ *Always check spill-to-disk metrics first — they indicate memory pressure, not just slow logic.*

Q2

You have a DataFrame with 500 million rows. After a `groupBy().agg()`, output has only 200 partitions but downstream operations are slow. What is happening and how do you fix it?

■ Asked at: LinkedIn · Google · Amazon | ■ Difficulty: Hard

■ **Answer:**

The `groupBy().agg()` shuffle produces partitions sized by `spark.sql.shuffle.partitions` (default 200). With 500M rows, each partition may hold millions of records causing OOM or slow tasks. Fix: (1) Increase shuffle partitions proportionally to data size. (2) With AQE enabled, use `spark.sql.adaptive.coalescePartitions.enabled=true` to auto-merge small ones. (3) After aggregation, use `repartition(n)` or `coalesce(n)` strategically. (4) If downstream is a write, use `partitionBy()` on a low-cardinality column to align physical layout with query patterns.

■ **Key Code / Config:**

```
spark.conf.set('spark.sql.shuffle.partitions', '2000')
result = df.groupBy('user_id').agg(F.sum('amount'))
# Repartition for downstream efficiency
result.repartition(500).write.parquet('output/')
```

■ *Rule of thumb: target 100–200 MB per partition. Calculate: $\text{total_shuffle_bytes} / \text{target_partition_size}$.*

Q3

A pipeline uses `.cache()` extensively but memory usage keeps growing until executors OOM. Describe what is wrong and the correct caching strategy.

■ Asked at: Meta · Airbnb · Twitter | ■ Difficulty: Hard

■ **Answer:**

Common mistakes: (1) Caching DataFrames derived from wide transformations without unpersisting — each `.cache()` pins that lineage. (2) Using `MEMORY_AND_DISK` when data exceeds executor memory causes excessive GC. (3) Not calling `.unpersist()` after the cached DF is no longer needed. Correct strategy: (a) Only cache DFs reused 2+ times. (b) Choose storage level based on reuse frequency and size — `MEMORY_ONLY` for hot small DFs, `MEMORY_AND_DISK_SER` for large ones. (c) Always unpersist explicitly. (d) Monitor via `spark.catalog.clearCache()` in tests. (e) Prefer checkpointing for iterative ML loops over caching.

■ **Key Code / Config:**

```
df_cached = df.persist(StorageLevel.MEMORY_AND_DISK_SER)
# ... multiple uses ...
df_cached.unpersist() # Always clean up

# For iterative algorithms, prefer checkpoint
sc.setCheckpointDir('hdfs://checkpoints/')
df.checkpoint()
```

■ Use Spark UI Storage tab to see fraction cached and memory consumed per RDD.

Q4

Your Spark SQL query on Delta Lake is slow despite Z-ordering. Explain when Z-ordering helps and when it doesn't, and what alternatives exist.

■ Asked at: Databricks · Snowflake · Palantir | ■ Difficulty: Medium

■ **Answer:**

Z-ordering co-locates related data in files to enable data skipping via min/max statistics. It helps when: (1) Queries filter on the Z-ordered columns with high selectivity. (2) Data isn't already clustered on those columns. It doesn't help when: (1) Cardinality is too low (e.g., boolean columns). (2) Filters use non-Z-ordered columns. (3) Table is small enough to scan fully. Alternatives: (a) Liquid Clustering (Delta 3.1+) — auto-manages clustering without full rewrites. (b) Partition pruning for high-cardinality date/region columns. (c) Bloom filters for exact-match lookups on high-cardinality string columns.

■ **Key Code / Config:**

```
-- Z-order on query-critical columns
OPTIMIZE events ZORDER BY (user_id, event_date);

-- Bloom filter for string lookups
ALTER TABLE events SET TBLPROPERTIES (
  'delta.dataSkippingNumIndexedCols' = '32',
  'delta.bloomFilter.columns' = 'session_id'
);
```

■ Z-ordering is most effective when combined with partition pruning on a coarser column (e.g., partition by date, Z-order by user_id).

Q5

You notice that a Spark job's executor GC time is >30% of total task time. What are the root causes and remediation steps?

■ Asked at: Uber · DoorDash · Lyft | ■ Difficulty: Hard

■ **Answer:**

High GC indicates memory pressure — too many objects are being created and promoted to Old Gen. Root causes: (1) Large number of small Java objects (e.g., Row objects from RDD API). (2) Insufficient executor memory relative to data being processed. (3) Using Kryo serialization is not enabled. (4) String interning or accumulator misuse creating object retention. Remediation: (a) Switch to Dataset/DataFrame API — Tungsten's off-heap columnar format reduces GC pressure. (b) Enable Kryo: `spark.serializer=org.apache.spark.serializer.KryoSerializer`. (c) Increase `spark.executor.memory` or decrease `spark.memory.fraction` to give more heap to user memory. (d) Use off-heap memory: `spark.memory.offHeap.enabled=true`. (e) Tune GC: use G1GC with `-XX:+UseG1GC`.

■ **Key Code / Config:**

```
spark.conf.set('spark.serializer', 'org.apache.spark.serializer.KryoSerializer')
spark.conf.set('spark.memory.offHeap.enabled', 'true')
spark.conf.set('spark.memory.offHeap.size', '4g')
# JVM options in spark-defaults.conf:
# spark.executor.extraJavaOptions -XX:+UseG1GC -XX:G1HeapRegionSize=16m
```

■ *Check Spark UI Executor tab for GC time column. Target <5% GC time.*

Q6

How do you tune Spark for a job that reads 1 TB of JSON, transforms it, and writes Parquet? Walk through the complete configuration strategy.

■ Asked at: Amazon · Google · Microsoft | ■ Difficulty: Hard

■ **Answer:**

Step-by-step: (1) Parallelism on read: JSON doesn't split well — use `spark.sql.files.maxPartitionBytes=128MB` and ensure enough cores. Prefer pre-converting to Parquet/Avro. (2) Set shuffle partitions: approx 1000-2000 for 1 TB. (3) Enable AQE for dynamic optimization. (4) Broadcast small lookup tables (<10 MB). (5) Use columnar Parquet output with snappy/zstd compression. (6) Executor sizing: 4-8 cores per executor, 16-32 GB memory, not too many small executors. (7) Enable speculative execution for straggler tasks. (8) For AWS EMR/cloud: use instance storage for shuffle (not EBS) via `spark.local.dir`.

■ **Key Code / Config:**

```
spark.conf.set('spark.sql.files.maxPartitionBytes', str(128*1024*1024))
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.shuffle.partitions', '1500')
spark.conf.set('spark.speculation', 'true')

df.write.option('compression', 'zstd') \
    .partitionBy('date') \
    .parquet('s3://bucket/output/')
```

■ *zstd compression gives better ratio than snappy with similar decompression speed — prefer it for cold storage.*

Q7**What is the Tungsten execution engine and how does it differ from the legacy RDD execution model? When does it not help?**

■ Asked at: Netflix · Spotify · Stripe | ■ Difficulty: Medium

■ **Answer:**

Tungsten is Spark's physical execution layer that bypasses JVM object overhead using: (1) Off-heap binary memory management — stores data in compact binary format, eliminating Java object header overhead (~16 bytes/object). (2) Cache-aware computation — sort/hash algorithms designed for CPU cache locality. (3) Whole-stage code generation (WSCG) — compiles multiple operators into a single JVM function, eliminating virtual function calls. Differences from RDD: RDD uses Java serialization and per-record function calls; Tungsten uses vectorized columnar processing. Tungsten doesn't help when: (a) Using Python UDFs — data must serialize to/from Python. (b) Using RDD API directly. (c) Complex Hive UDFs that bypass Catalyst.

■ **Key Code / Config:**

```
# Tungsten WSCG can be checked via explain
df.explain(mode='codegen')

# Disable to debug (not for production)
spark.conf.set('spark.sql.codegen.wholeStage', 'false')
```

■ *Replace Python UDFs with Pandas UDFs (vectorized) to keep data in Arrow format and minimize Python serialization overhead.*

Q8**Explain Adaptive Query Execution (AQE). Describe three real-world scenarios where AQE significantly improves performance.**

■ Asked at: Databricks · Cloudera · Hortonworks | ■ Difficulty: Hard

■ **Answer:**

AQE re-optimizes the query plan at runtime using actual statistics collected after each shuffle stage, rather than relying on estimated stats. Three key features: (1) Dynamic partition coalescing — after a shuffle, if actual partition sizes are small, AQE merges them to avoid too many tiny tasks. Scenario: after aggressive filtering, 2000 shuffle partitions become 50 MB total → merged to 25 partitions. (2) Dynamic skew join handling — detects skewed partitions (>spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes) and splits them. Scenario: user_id='unknown' has 50% of data → split into sub-partitions automatically. (3) Dynamic join strategy switching — converts SortMergeJoin to BroadcastHashJoin when the build side turns out small after filtering. Scenario: filtered table drops from 10 GB to 50 MB → auto-broadcast.

■ **Key Code / Config:**

```
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.adaptive.skewJoin.enabled', 'true')
spark.conf.set('spark.sql.adaptive.coalescePartitions.enabled', 'true')
spark.conf.set('spark.sql.adaptive.skewJoin.skewedPartitionFactor', '5')
spark.conf.set('spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes', '256MB')
```

■ *AQE requires at least one shuffle stage to collect runtime statistics. It cannot optimize plans with no shuffle.*

Section 2: Memory Management & Spill

Q9**Describe Spark's Unified Memory Model. How does it allocate memory between execution and storage, and what happens during memory pressure?**

■ Asked at: Uber · Lyft · Grab | ■ Difficulty: Hard

■ **Answer:**

Spark's Unified Memory Model (introduced in 1.6) divides JVM heap into: (1) Reserved Memory (~300 MB) — for Spark internals. (2) User Memory $(1 - \text{spark.memory.fraction}) * \text{heap}$ — for user data structures, not managed by Spark. (3) Spark Memory ($\text{spark.memory.fraction}$, default 0.6) — split between Execution (shuffles, sorts, joins) and Storage (cached data). The key innovation is the unified boundary — execution can borrow from storage memory and vice versa. Under memory pressure: execution memory can evict cached blocks from storage. However, execution memory cannot be evicted once in use (it would cause incorrect results). Storage memory can only borrow execution memory when execution pool is idle. Result: if both are under pressure, storage evicts to disk, execution spills to disk.

■ **Key Code / Config:**

```
# Memory fractions
spark.conf.set('spark.memory.fraction', '0.6')           # Spark-managed pool
spark.conf.set('spark.memory.storageFraction', '0.5')    # Initial storage within Spark pool
# Total executor memory budget:
# spark.executor.memory + spark.executor.memoryOverhead (for off-heap, Python, etc.)
```

■ *For shuffle-heavy workloads, lower `spark.memory.storageFraction` to give execution more initial memory.*

Q10**You see 'spill to disk' warnings in your Spark job. Explain the different types of spill and the performance impact of each.**

■ Asked at: Meta · Amazon · Pinterest | ■ Difficulty: Hard

■ **Answer:**

Two types of spill: (1) Shuffle spill — occurs during sort/hash operations when the in-memory buffer is full. Data is sorted and written to disk in multiple sorted runs, then merged on read. Measured in Spark UI as 'Shuffle Write Spill' and 'Shuffle Read Spill'. (2) Storage spill — when cached DataFrames with MEMORY_AND_DISK storage level evict partitions to disk due to memory pressure. Performance impact: disk I/O is 10-100x slower than memory. If spill is significant, total job time can increase 3-5x. Remediation: (a) Increase executor memory. (b) Reduce partition size by increasing shuffle partition count. (c) Enable off-heap memory to reduce GC interference. (d) For aggregations, use `reduceByKey` instead of `groupByKey` (pre-aggregates before shuffle). (e) Consider bucketing to eliminate shuffle altogether.

■ **Key Code / Config:**

```
# Reduce spill by tuning partition size
spark.conf.set('spark.sql.shuffle.partitions', '4000')

# Monitor spill in Spark UI or programmatically
# Look for 'spill (memory)' and 'spill (disk)' columns in Stages tab
```

■ *A small amount of spill is acceptable. Alarm when spill (disk) > 10% of total shuffle write bytes.*

Q11

When would you use off-heap memory in Spark? What are the risks and trade-offs?

■ Asked at: Cloudera · Databricks · IBM | ■ Difficulty: Hard

■ Answer:

Off-heap memory lives outside the JVM heap — allocated via `sun.misc.Unsafe` and managed by Spark/Tungsten directly. When to use: (1) Very large executor memory (>32 GB) where GC pauses become significant. (2) Caching massive DataFrames that cause frequent GC. (3) High-frequency columnar operations benefiting from CPU cache locality. Trade-offs: Benefits: eliminates GC overhead for managed data, more predictable performance, can exceed JVM heap limits. Risks: (a) Memory leaks are harder to detect — no GC collection. (b) Native OOM errors bypass JVM error handling and can crash executors. (c) Debugging is harder — standard Java profilers don't see off-heap. (d) Interaction with Python UDFs still requires JVM serialization. Configuration: `spark.memory.offHeap.enabled` + `spark.memory.offHeap.size`.

■ Key Code / Config:

```
spark.conf.set('spark.memory.offHeap.enabled', 'true')
spark.conf.set('spark.memory.offHeap.size', '8g') # Per executor

# Also increase memoryOverhead for container managers
# spark.executor.memoryOverhead = max(384MB, 0.1 * executor.memory)
```

■ *When using Kubernetes or YARN, always account for off-heap in container memory limits to avoid container kills.*

Q12

Your Spark executors are being killed by YARN/Kubernetes with 'Container killed on request' or OOMKilled. Walk through the diagnosis and fix.

■ Asked at: Netflix · Spotify · Twilio | ■ Difficulty: Medium

■ Answer:

Container kills happen when process memory exceeds container allocation. Two scenarios: (1) JVM heap OOM: task too large for executor heap → increase `spark.executor.memory`. (2) Off-heap OOM: Python processes, native libraries, shuffle buffers exceeding overhead → increase `spark.executor.memoryOverhead` (YARN) or `spark.kubernetes.memoryOverheadFactor` (K8s). Diagnosis steps: (a) Check container logs for OOMKilled signal. (b) Check Spark UI Executor tab for GC time and peak memory usage. (c) If Python UDFs used, Python process memory is not tracked by JVM. (d) Check for large broadcasts — `spark.broadcast.blockSize` and total broadcast size. Fix: Systematically increase overhead first (cheaper), then executor memory. For persistent OOMs, reduce `spark.executor.cores` to reduce concurrent task memory pressure.

■ Key Code / Config:

```
# YARN
spark.conf.set('spark.executor.memoryOverhead', '2g')

# Kubernetes
spark.conf.set('spark.kubernetes.memoryOverheadFactor', '0.2') # 20% of executor memory

# Reduce concurrent task pressure
spark.conf.set('spark.executor.cores', '4') # Not 8+ per executor
```

■ *Total container memory = `spark.executor.memory` + `spark.executor.memoryOverhead`. Request this as container size.*